

Aim:- Naive Bayes' Classifier

- Implement the Naive Bayes' algorithm for classification.
- Train a Naive Bayes' model using a given dataset and calculate class probabilities.
- Evaluate the accuracy of the model on test data and analyze the results.

Code:-

```
Practical 7.py - D:\Haripr\A.I\Practical 7.py (3.13.5)
File Edit Format Run Options Window Help
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder, label_binarize
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report, roc_curve, auc
print("Hariprasad Vishwakarma T127")
df = pd.read_csv(r"F:\Downloads\Most Runs - 2016.csv")
print("Dataset Loaded Successfully!")
print("\nDataset Shape:")
print(df.shape)
print("\nFirst 5 Rows:")
print(df.head())

if "Player" in df.columns:
    df = df.drop("Player", axis=1)
if "POS" in df.columns:
    df = df.drop("POS", axis=1)

if "HS" in df.columns:
    df["HS"] = df["HS"].astype(str).str.replace("*", "", regex=False)
    df["HS"] = pd.to_numeric(df["HS"], errors="coerce")

for column in df.columns:
    df[column] = pd.to_numeric(df[column], errors="coerce")

df = df.dropna()
print("\nMissing values removed")

def performance_category(runs):
    if runs <= 250:
        return "Low"
    elif runs < 450:
        return "Medium"
    else:
        return "High"

df["Performance_Category"] = df["Runs"].apply(performance_category)

print("\nPerformance Categories:")
print(df["Performance_Category"].value_counts())
```

MVLU COLLEGE

A.I. Practical No.7

```
Practical 7.py - D:\Har\AI\Practical 7.py (3.11.5)
File Edit Format Run Options Window Help
print("\nPerformance Categories:")
print(df["Performance_Category"].value_counts())

X = df.drop(["Runs", "Performance_Category"], axis=1)
y = df["Performance_Category"]

encoder = LabelEncoder()
y = encoder.fit_transform(y)

print("\nClasses:")
for i, name in enumerate(encoder.classes_):
    print(i, "=", name)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)

print("\nTraining Samples:")
print(len(X_train))
print("\nTesting Samples:")
print(len(X_test))

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

print("\nData Standardized Successfully!")

model = GaussianNB()
model.fit(X_train, y_train)

print("\nGaussian Naive Bayes Model Trained Successfully!")

y_pred = model.predict(X_test)

print("\nPredicted Classes:")
print(encoder.inverse_transform(y_pred))

accuracy = accuracy_score(y_test, y_pred)

print("\nAccuracy:")
print(accuracy)

print("\nAccuracy:")
print(accuracy)
print("\nAccuracy Percentage:")
print(round(accuracy * 100, 2), "%")

cm = confusion_matrix(y_test, y_pred)

print("\nConfusion Matrix:")
print(cm)

print("\nClassification Report:")
print(classification_report(
    y_test,
    y_pred,
    target_names=encoder.classes_,
    zero_division=0
))

y_prob = model.predict_proba(X_test)

print("\nPrediction Probabilities:")
print(y_prob)

y_test_binary = label_binarize(
    y_test,
    classes=range(len(encoder.classes_))
)

# Create Confusion Matrix Figure
plt.figure(figsize=(7, 6))
plt.imshow(cm)
plt.title("Confusion Matrix - IPL 2016 Player Performance")
plt.xlabel("Predicted Class")
plt.ylabel("Actual Class")
plt.xticks(range(len(encoder.classes_)), encoder.classes_)
plt.yticks(range(len(encoder.classes_)), encoder.classes_)

for i in range(len(cm)):
    for j in range(len(cm)):
        plt.text(j, i, cm[i, j], ha="center", va="center")

plt.colorbar()
```

MVLU COLLEGE

A.I. Practical No.7

```
*Practical 7.py - D:/Hari/A.I/Practical 7.py (3.13.5)
File Edit Format Run Options Window Help

plt.colorbar()
plt.tight_layout()

# Create ROC Curve Figure
plt.figure(figsize=(8, 6))

for i in range(len(encoder.classes_)):
    if len(set(y_test_binary[:, i])) == 2:
        fpr, tpr, _ = roc_curve(
            y_test_binary[:, i],
            y_prob[:, i]
        )
        roc_auc = auc(fpr, tpr)
        plt.plot(
            fpr,
            tpr,
            label=encoder.classes_[i] +
            " (AUC = " +
            str(round(roc_auc, 2)) +
            ")"
        )

plt.plot(
    [0, 1],
    [0, 1],
    linestyle="--",
    label="Random Classifier"
)

plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve - Gaussian Naive Bayes")
plt.legend()
plt.grid()
plt.tight_layout()
# Display all graphs
plt.show()
print("\n=====")
print("NAIVE BAYES PRACTICAL COMPLETED")
print("=====")
print("Accuracy:", round(accuracy * 100, 2), "%")
print("=====")
```

Output:-

```
*IDLE Shell 3.13.5*
File Edit Shell Debug Options Window Help

Python 3.13.5 (tags/v3.13.5:6cb20a2, Jun 11 2025, 16:15:46) [MSC v.1943 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
===== RESTART: D:/Hari/A.I/Practical 7.py =====
Hariprasad Vishwakarma T127
Dataset Loaded Successfully!

Dataset Shape:
(136, 14)

First 5 Rows:
   POS  Player  Mat  Inns  NO  Runs  ...  BF    SR   100  50  4s  6s
0    1   Virat Kohli  16   16  4   973  ...  640  152.03  4   7   83  38
1    2   David Warner  17   17  3   848  ...  560  151.42  0   9   88  31
2    3   AB de Villiers  16   16  3   687  ...  407  168.79  1   6   57  37
3    4   Shikhar Dhawan  17   17  4   501  ...  429  116.78  0   4   51   8
4    5   Gautam Gambhir  15   15  2   501  ...  411  121.89  0   5   54   6

[5 rows x 14 columns]

Missing values removed

Performance Categories:
Performance_Category
Low      112
Medium   16
High      8
Name: count, dtype: int64

Classes:
0 = High
1 = Low
2 = Medium

Training Samples:
108

Testing Samples:
28

Data Standardized Successfully!
```

Hariprasad Vishwakarma
T127
TYCS

MVLU COLLEGE

A.I. Practical No.7

```
"DLE Shell 3.13.5"
File Edit Shell Debug Options Window Help
Data Standardized Successfully!

Gaussian Naive Bayes Model Trained Successfully!

Predicted Classes:
['Low' 'Low' 'Low' 'Low' 'Medium' 'Low' 'Low' 'Low' 'Low' 'Low'
'Low' 'High' 'Low' 'Low' 'Low' 'Low' 'Low' 'Low' 'Medium' 'Low'
'High' 'Low' 'Medium' 'Low' 'Low' 'Low']

Accuracy:
1.0

Accuracy Percentage:
100.0 %

Confusion Matrix:
[[ 2  0  0]
 [ 0 23  0]
 [ 0  0 3]]

Classification Report:
              precision    recall  f1-score   support

      High         1.00        1.00        1.00         2
      Low          1.00        1.00        1.00        23
      Medium       1.00        1.00        1.00         3

   accuracy         1.00
  macro avg         1.00
 weighted avg         1.00

Prediction Probabilities:
[[7.74177524e-020 9.99999999e-001 6.67762092e-010]
 [3.78079148e-039 1.00000000e+000 2.15445228e-018]
 [1.32990274e-042 9.99999999e-001 5.36451434e-010]
 [2.30226624e-065 1.00000000e+000 1.23172403e-017]
 [2.39786646e-006 9.87848853e-015 9.99997602e-001]
 [3.43439382e-087 1.00000000e+000 3.17392000e-045]
 [5.39214875e-017 9.99999707e-001 2.93356154e-007]
 [4.13484296e-086 1.00000000e+000 1.91551397e-031]
 [5.93851210e-023 9.98513466e-001 1.48653408e-003]
 [1.27768617e-041 1.00000000e+000 1.36775180e-019]
 [7.49227943e-052 1.00000000e+000 1.96693542e-030]
 [1.78805350e-087 1.00000000e+000 4.32164560e-066]
 [9.23114155e-001 1.07130923e-042 7.68858453e-002]
 [5.58355159e-026 1.00000000e+000 6.87325029e-011]
 [5.68565010e-063 1.00000000e+000 1.48080722e-032]
 [8.14806500e-099 1.00000000e+000 3.26983279e-059]
 [1.04642400e-033 1.00000000e+000 2.01831087e-012]
 [1.95211847e-096 1.00000000e+000 1.66966540e-042]
 [2.55600993e-082 1.00000000e+000 4.08396178e-031]
 [1.19032253e-066 1.00000000e+000 3.22784093e-029]
 [9.39350279e-006 1.59241221e-011 9.99990606e-001]
 [1.11429375e-057 1.00000000e+000 1.82582887e-024]
 [9.93712229e-001 3.41870039e-030 6.28777058e-003]
 [1.37609535e-108 1.00000000e+000 2.02915781e-050]
 [2.37970421e-010 2.67864195e-007 9.9999732e-001]
 [2.42008642e-051 1.00000000e+000 1.11447642e-021]
 [9.23051849e-085 1.00000000e+000 3.96293415e-048]
 [5.91707705e-114 1.00000000e+000 1.25481289e-065]]

Classification Report:
              precision    recall  f1-score   support

      High         1.00        1.00        1.00         2
      Low          1.00        1.00        1.00        23
      Medium       1.00        1.00        1.00         3

   accuracy         1.00
  macro avg         1.00
 weighted avg         1.00

Prediction Probabilities:
[[7.74177524e-020 9.99999999e-001 6.67762092e-010]
 [3.78079148e-039 1.00000000e+000 2.15445228e-018]
 [1.32990274e-042 9.99999999e-001 5.36451434e-010]
 [2.30226624e-065 1.00000000e+000 1.23172403e-017]
 [2.39786646e-006 9.87848853e-015 9.99997602e-001]
 [3.43439382e-087 1.00000000e+000 3.17392000e-045]
 [5.39214875e-017 9.99999707e-001 2.93356154e-007]
 [4.13484296e-086 1.00000000e+000 1.91551397e-031]
 [5.93851210e-023 9.98513466e-001 1.48653408e-003]
 [1.27768617e-041 1.00000000e+000 1.36775180e-019]
 [7.49227943e-052 1.00000000e+000 1.96693542e-030]
 [1.78805350e-087 1.00000000e+000 4.32164560e-066]
 [9.23114155e-001 1.07130923e-042 7.68858453e-002]
 [5.58355159e-026 1.00000000e+000 6.87325029e-011]
 [5.68565010e-063 1.00000000e+000 1.48080722e-032]
 [8.14806500e-099 1.00000000e+000 3.26983279e-059]
 [1.04642400e-033 1.00000000e+000 2.01831087e-012]
 [1.95211847e-096 1.00000000e+000 1.66966540e-042]
 [2.55600993e-082 1.00000000e+000 4.08396178e-031]
 [1.19032253e-066 1.00000000e+000 3.22784093e-029]
 [9.39350279e-006 1.59241221e-011 9.99990606e-001]
 [1.11429375e-057 1.00000000e+000 1.82582887e-024]
 [9.93712229e-001 3.41870039e-030 6.28777058e-003]
 [1.37609535e-108 1.00000000e+000 2.02915781e-050]
 [2.37970421e-010 2.67864195e-007 9.9999732e-001]
 [2.42008642e-051 1.00000000e+000 1.11447642e-021]
 [9.23051849e-085 1.00000000e+000 3.96293415e-048]
 [5.91707705e-114 1.00000000e+000 1.25481289e-065]]
```

MVLU COLLEGE

A.I. Practical No.7

Figure 1

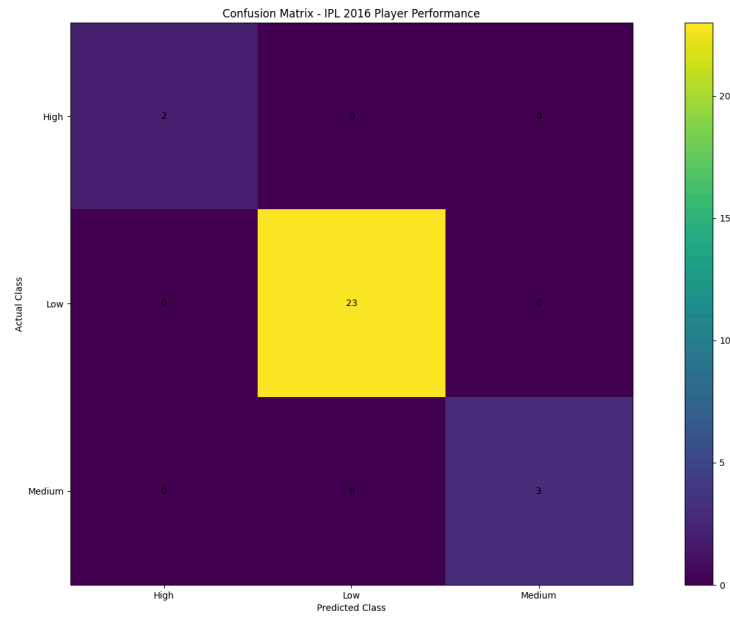
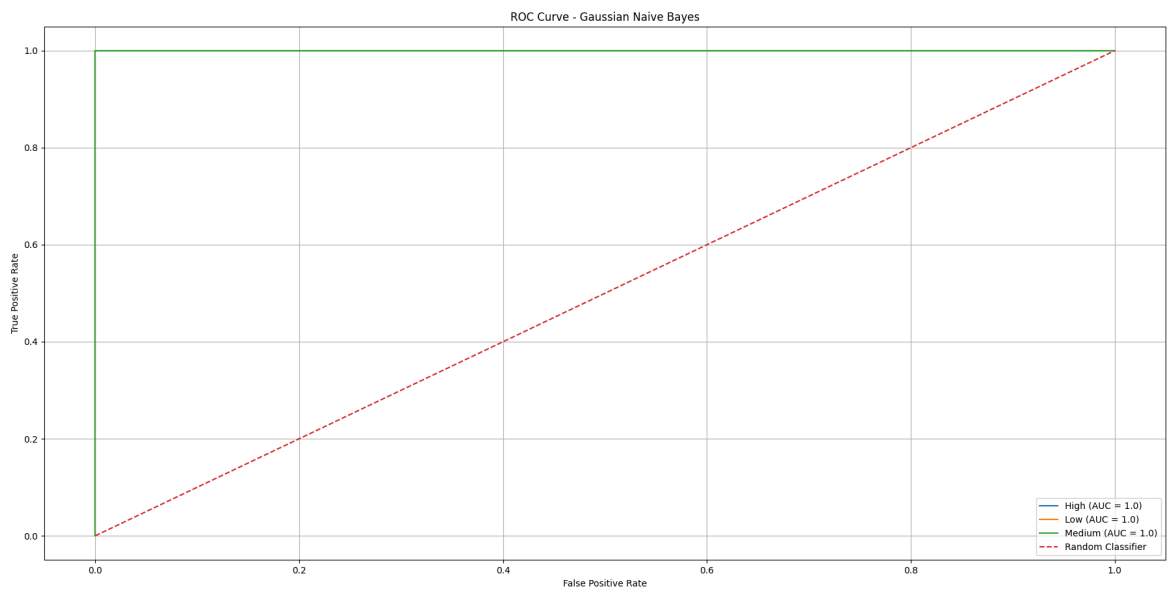


Figure 2



MVLU COLLEGE

A.I. Practical No.7

```
IDE Shell 3.13.5
File Edit Shell Debug Options Window Help

accuracy          1.00          1.00          28
macro avg         1.00          1.00          28
weighted avg      1.00          1.00          28

Prediction Probabilities:
[[7.74177524e-020 9.99999999e-001 6.67762092e-010]
[3.78079148e-039 1.00000000e+000 2.15445228e-018]
[1.32990274e-042 9.99999999e-001 5.36451434e-010]
[2.30226624e-065 1.00000000e+000 1.23172403e-017]
[2.39786646e-006 9.87848853e-015 9.9997602e-001]
[3.43439382e-087 1.00000000e+000 3.17392000e-045]
[5.39214875e-017 9.9999707e-001 2.93356154e-007]
[4.13484296e-086 1.00000000e+000 1.91551397e-031]
[5.93851210e-023 9.98513466e-001 1.48653408e-003]
[1.27768617e-041 1.00000000e+000 1.36775180e-019]
[7.49227943e-052 1.00000000e+000 1.96693542e-030]
[1.78805350e-087 1.00000000e+000 4.32164560e-066]
[9.23114155e-001 1.07130923e-042 7.68858453e-002]
[5.58355159e-026 1.00000000e+000 6.87325029e-011]
[5.68565010e-063 1.00000000e+000 1.48080722e-032]
[8.14806500e-099 1.00000000e+000 3.26983279e-059]
[1.04642400e-033 1.00000000e+000 2.01831087e-012]
[1.95211847e-096 1.00000000e+000 1.66966540e-042]
[2.55600993e-082 1.00000000e+000 4.08396178e-031]
[1.19032253e-066 1.00000000e+000 3.22784093e-029]
[9.39350279e-006 1.59241221e-011 9.99990606e-001]
[1.11429375e-057 1.00000000e+000 1.82582887e-024]
[9.93712229e-001 3.41870039e-030 6.28777058e-003]
[1.37609535e-108 1.00000000e+000 2.02915781e-050]
[2.37970421e-010 2.67864195e-007 9.9999732e-001]
[2.42008642e-051 1.00000000e+000 1.11447642e-021]
[9.23051849e-085 1.00000000e+000 3.96293415e-048]
[5.91707705e-114 1.00000000e+000 1.25481289e-065]]

NAIVE BAYES PRACTICAL COMPLETED
Accuracy: 100.0 %

>>>
```